

R2LSH: A Nearest Neighbor Search Scheme Based on Two-dimensional Projected Spaces

Kejing Lu* Mineichi Kudo*

*Graduate School of Information Science and Technology, Hokkaido University, Japan
{lkejing@ist.hokudai.ac.jp, mine@ist.hokudai.ac.jp}

Abstract—Locality sensitive hashing (LSH) is a widely practiced c -approximate nearest neighbor (c -ANN) search algorithm because of its appealing theoretical guarantee and empirical performance. However, available LSH-based solutions do not achieve a good balance between cost and quality because of certain limitations in their index structures.

In this paper, we propose a novel and easy-to-implement disk-based method named R2LSH to answer ANN queries in high-dimensional spaces. In the indexing phase, R2LSH maps data objects into multiple two-dimensional projected spaces. In each space, a group of B+-trees is constructed to characterize the corresponding data distribution. In the query phase, by setting a query-centric ball in each projected space and using a dynamic counting technique, R2LSH efficiently determines candidates and returns query results with the required quality. Rigorous theoretical analysis reveals that the proposed algorithm supports c -ANN search for arbitrarily small $c \geq 1$ with probability guarantee. Extensive experiments on real datasets verify the superiority of R2LSH over state-of-the-art methods.

Index Terms—Locality Sensitive Hashing, Approximate Nearest Neighbor Search, High-Dimensional Spaces

I. INTRODUCTION

The nearest neighbor (NN) search in a high-dimensional Euclidean space is of substantial importance in areas such as database, information retrieval, data mining, pattern recognition, and machine learning [6], [11]. The common approach to overcoming the problem of dimensionality is to design efficient c -approximate NN search algorithms by compromising on precision for speed [2]. A point o is called a c -approximate NN (c -ANN) of a query q if its distance to q is at most c times the distance from q to its exact NN o^* , i.e., $\|q, o\| \leq c\|q, o^*\|$. Here, $c (\geq 1)$ is an *approximation ratio*. As a highly potential c -ANN search algorithm, locality sensitive hashing (LSH) displays effective query performance and success probability guarantee in theory [9]. Moreover, it has broad applications in practice [11].

The original LSH method, E2LSH [3], incurs prohibitively large storage cost for each data object to be stored in multiple index structures [16]. Several LSH variants such as LSH-forest [16], C2LSH [5], SRS [15], and QALSH [8] have been proposed to mitigate the cost and answer c -ANN queries with reasonably small indexes and constant success probability.

Among these LSH variants, QALSH is state-of-the-art and displays stable performances on various datasets. In the indexing phase, QALSH randomly generates m hash functions $\{h_{a_i}\}_{i=1}^m$ (where $\{a_i\}$ are projection vectors) and computes hash values of each object o by $h_{a_i}(o) = \langle a_i, o \rangle$ ($1 \leq i \leq m$).

In the query phase, on the line spanned by a_i , QALSH sets up a query-centric hash bucket $[h_{a_i}(q) - w, h_{a_i}(q) + w]$. Here, q is the query. w denotes the half-width of the hash bucket and is initially set to zero. During the search process, QALSH gradually increases w and scans those objects colliding with q in the hash buckets. By the dynamic counting technique [5], QALSH updates the collision frequency (number) $\#Col(o)$ of each object o as w is increased. When a certain object o passes the collision testing, i.e., $\#Col(o)$ attains a specified threshold τ , this object is considered as a candidate, and its distance to q is computed accurately. After the search terminates, the nearest neighbors of q among the candidates are returned to the users.

QALSH can remove most false objects by collision testing. Thus, the size of the candidate set is fairly small, which significantly reduces the I/O cost of retrieving candidates. However, before identifying candidates, QALSH consumes a significant amount of I/Os to scan those objects colliding with q owing to the inefficiency of one-dimensional hash buckets. Consider Figure 1(a) as an example. For simplicity, we suppose there are only two hash functions h_{a_1} and h_{a_2} . The threshold τ is set to two, which implies that only those objects that collide with q in two hash buckets simultaneously can be considered as candidates (red points). To obtain these candidates, QALSH has to scan all the objects (black points) that lie between two hyperplanes at a distance of $2w$ from each other. Because the number of scanned objects can be exceptionally large and these objects may appear in multiple hash buckets, the scanning process incurs substantial I/O cost. Almost all the I/O cost incurred in the query phase of QALSH originates from this scanning.

Thus, to reduce the I/O cost of scanning, we propose a more efficient indexing and searching scheme named R2LSH. The fundamental concept underlying R2LSH is illustrated in Figure 1(b). Similarly, we again consider the case of the two hash functions h_{a_1} and h_{a_2} . First, using the mapping $o \mapsto (h_{a_1}(o), h_{a_2}(o))$ (where o is an arbitrary object in the dataset), we can construct a two-dimensional space M that contains images of all the objects, such that the coordinate of the mapped o in M is $(h_{a_1}(o), h_{a_2}(o))$. Then, rather than use two hash buckets, we construct a query-centric ball of radius r in this space and ensure that all the objects within this ball can be scanned by introducing a suitable index structure (a small number of objects outside the ball may also be scanned). All the scanned objects are considered as candidates (red points). Evidently, query-centric balls work more selectively

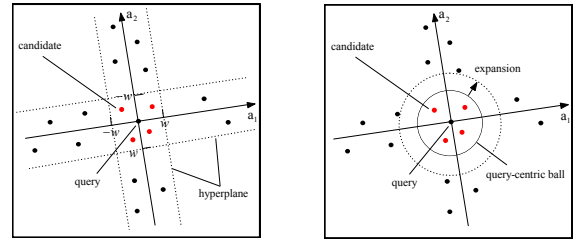
than query-centric hash buckets because only those objects whose projections are close to the query's projections on both the axes are likely to be scanned. Therefore, we can circumvent substantial I/O cost in scanning false objects, which also leads to a smaller m (around 40) of R2LSH than that of QALSH (around or over 100) in practice.

Based on this concept, in the indexing phase, R2LSH generates $2m$ hash functions and constructs m two-dimensional projected spaces into which data objects are mapped. In each projected space, a number of rings are constructed such that each object is contained in exactly one of these rings. By introducing the polar coordinate system, objects in the same ring can be indexed by a B+-tree based on their angle coordinates (the ring's center is considered as the origin). In the query phase, a query-centric ball is set up in each projected space with a uniform radius r . When o falls inside the ball, we consider it as o colliding with q in that ball. As the search proceeds, we expand all the balls at an equal rate and dynamically update the collision number of each data object. Since all objects are contained in these rings, such update can be realized by considering the position relationship between the query-centric ball and each ring. According to the collision numbers of data objects, we determine candidates to calculate the distances between them and query. When the termination condition is satisfied, we terminate the expansion of the balls and return the results from the candidates.

Notwithstanding the apparent advantage of query-centric balls, the realization of R2LSH is non-trivial because two crucial problems need to be solved: (1) how to control the radii of query-centric balls, and (2) how to efficiently identify those objects falling inside balls. In Secs. V and VI, we illustrate our solution in detail. An additional aspect that needs attention is that query-centric balls of higher dimensions (> 2) or those using other tree-structures appear to be available in the above-mentioned process as well. However, we do not do so because (1) in our searching scheme, B+-trees are superior to other tree structures in terms of time consumption owing to the sequential scanning in index access, and (2) objects in a two-dimensional ring can be indexed by a B+-tree naturally and efficiently. In Sec VIII-B, we use a few experiments to support these two statements.

Our main contributions are summarized as follows:

- We propose an efficient indexing and searching scheme called R2LSH to answer approximate NN queries with theoretical guarantee in high-dimensional Euclidean spaces. Parameter tuning of R2LSH is more or less convenient, which is practically important for supporting different approximation ratios and different success probabilities specified by users.
- In the query phase, R2LSH determines candidates by means of query-centric balls in a group of two-dimensional projected spaces. Compared with traditional hash buckets, such balls are more selective and can efficiently reduce the likelihood that false objects are scanned. This results in a significant improvement in the performance.



(a) Hash bucket

(b) Ball (dim 2)

Figure 1: Query-centric Hash bucket ($m = 2$, $\tau = 2$) vs. Query-centric Ball ($\tau = 1$). We observe that for two query-centric hash buckets, each candidate (in red) should be scanned two times. In addition, many false objects (in black) are scanned once. When we use a two-dimensional query-centric ball, each candidate is scanned only one time. Because the ball is bounded, those far neighbors can be pruned efficiently. Thus, a query-centric ball can play the role of two hash buckets and substantially reduce the cost in scanning.

- Experimental results reveal that R2LSH generally requires less than 25% of the I/O cost (or running time) incurred by other LSH-based methods to achieve a similar precision level. This demonstrates the higher efficiency of R2LSH.

II. RELATED WORK

A. LSH Variants with Probability Guarantee

Among the approximate NN (ANN) search algorithms, LSH is the most widely used technique owing to its remarkable theoretical guarantees and empirical performance. E2LSH [4], the classical LSH implementations for ℓ_2 norm, cannot solve the c -ANN search problem efficiently because it requires a prohibitively large space in indexing. To reduce the storage cost, LSB-Forest [16] and C2LSH [5] use the so-called virtual rehashing technique, implicitly or explicitly, to prevent constructing physical hash tables for each search radius. Based on the concept of query-aware hashing, the two state-of-the-art algorithms QALSH and SRS improve the efficiency over that of C2LSH by using different index structures and search methods, respectively.

B. Other ANN Methods

All the aforementioned LSH algorithms provide probability guarantees on the result quality. In addition to these, there are other types of ANN methods as follows.

As a representative of learning-based methods, production quantization (PQ) divides the feature space into disjoint subspaces and then quantizes each subspace separately into multiple clusters [10]. By concatenating codes from different subspaces together, PQ partitions the feature space into a large number of fine-grained clusters, which enables efficient NN search. Another type of potential ANN methods is the graph-based algorithm. A representative method of this type, HNSW [14] uses long-range links to simulate the small-world property on the basis of an approximation of the

Delaunay graph. Although PQ and HNSW can achieve good performances on query accuracy, they exhibit the following three limitations: (1) they are memory-based methods and have difficulty working with large datasets in a commodity PC, as illustrated in [1]; (2) their high training cost (preprocessing overhead) is a barrier for very large datasets and updating [17], [18]; and (3) they are devoid of theoretical guarantees on query results. Recently, based on the idea of PQ, Liu *et al.* proposed an external-memory method called PQBF to conduct ANN search [13]. Although PQBF can be applied to disk-resident data, it still has the second and the third limitations of learning methods mentioned above.

HD-index [1] is a disk-based indexing scheme. It was proposed recently to answer approximate queries. HD-Index consists of a set of hierarchical structures called RDB-trees constructed on Hilbert keys of database objects. In the query phase, objects that are close to the query in an arbitrary RDB-tree are determined as candidates. Although HD-index can support very large datasets, it exhibits the following two limitations: (1) its performance in high-dimensional spaces is not adequate; and (2) owing to the absence of probability guarantee, it is challenging to increase its query accuracy to a user-specified level.

C. Benchmark Methods Selection

Referring to the classification of ANN methods in [12], we choose SRS and QALSH as our benchmark methods of LSH type because they are state-of-the-art methods with superiorities over LSH-forest and C2LSH belonging to the same category based on the empirical evidence in [8] and [15]. HD-index is chosen as the benchmark method of non-LSH type. For completion, we also choose PQBF as a representative of learning methods.

III. PRELIMINARIES

For a dataset $\mathcal{D}$ of n d -dimensional data points, an NN search identifies the point o^* in $\mathcal{D}$ with the minimum distance to a specified query q . For a c -ANN search with a constant c (≥ 1), only a c -approximate neighbor o needs to be returned. That is, $\|q - o\| \leq c\|q - o^*\|$. Here, $\|q - o\|$ denotes the ℓ_2 distance between q and o . A k NN search returns k results o_j^* ($1 \leq j \leq k$). Here, o_j^* is the j th nearest neighbor of q . Its c -approximate version, c - k -ANN, returns a set of k objects o_j ($1 \leq j \leq k$) satisfying $\|q - o_j\| \leq c\|q - o_j^*\|$.

Suppose a is a random projection vector each of whose entry is an i.i.d. random variable following the standard normal distribution $\mathcal{N}(0, 1)$. The inner product between a and vector o , denoted as $h(o) = \langle a, o \rangle$ is an LSH signature of o . We have the following important Lemma [4]:

Lemma 1. For any points o_1, o_2 in $\mathbb{R}^d$, $h(o_1) - h(o_2)$ follows the normal distribution $\mathcal{N}(0, \|o_1 - o_2\|^2)$.

Given a positive real number w , the interval $[h(q) - w, h(q) + w]$ is referred to as a *query-centric hash bucket*. For any point o , at distance s to q ($s = \|q - o\|$), the probability $p(s, w)$ that $h(o)$ falls in this bucket is expressed as follows:

$$p(s, w) = \Pr[|h(q) - h(o)| \leq w] = \int_{-\frac{w}{s}}^{\frac{w}{s}} \varphi(x) dx, \quad (1)$$

where $\varphi(x)$ is the probability density function (PDF) of $\mathcal{N}(0, 1)$.

Let $g(o) = (h_1(o), h_2(o), \dots, h_m(o))$ denote the random mapping of o from the *original* d -dimensional space to the m -dimensional *projected space* and $\Delta(o) = \|g(q) - g(o)\|$. Sun *et al.* [15] observed that $\Delta^2(o)/\|q - o\|^2$ follows the $\chi^2(m)$ distribution. Thus, as a generalization of (1), the probability $p(s, r)$ that $g(o)$ ($s = \|q - o\|$) falls within the hyperball of radius r centered at $g(q)$ is expressed as

$$p(s, r) = \Pr[\Delta(o) \leq r] = \Psi_m\left(\frac{r^2}{\|q - o\|^2}\right), \quad (2)$$

where $\Psi_m(x)$ is the cumulative distribution function (CDF) of the $\chi^2(m)$ distribution.

Table I: Notations

Notation	Explanation
d	the dimensionality of data object
c	the approximation ratio specified by users
δ	the error rate specified by users
o^*, o_i^*	the first and i th nearest neighbor of q
$\hat{o}, \hat{o}_i$	the first and i th nearest candidate of q
$s^*, \hat{s}$	$s^* = \ o^* - q\ $ and $\hat{s} = \ \hat{o} - q\ $
m	the number of generated projected spaces
M_i	the i th projected space ($1 \leq i \leq m$)
α	the number of reference objects in a single projected space
β	the number of rings centered at each reference object
$g_i(o)$	the image of o in M_i
$\Delta_i(o)$	$\Delta_i(o) = \ g_i(o) - g_i(q)\ $
$B(g_i(q), r)$	the query-centric ball centered at $g_i(q)$, of radius r
$\#Col(o; r)$	the collision number of o when the radii of query-centric balls are commonly set to r
C_j^i	the i th reference object in M_i
$R_{j,l}^i$	the l th ring centered at C_j^i in M_i
v_i	the reference vector in M_i
$T_{j,l}^i$	the B+-tree contains the information of objects in $R_{j,l}^i$
τ	the threshold on collision numbers
λ	$\lambda = \sqrt{\frac{1}{2m} \log \frac{1}{\delta} + \frac{\tau}{m}}$
Φ	CDF of the chi-square distribution $\chi^2(2)$

IV. OVERVIEW

An overview of R2LSH is depicted in Figure 2. In the indexing phase, we index all the data objects by multiple B+-trees as follows. First, we construct m two-dimensional projected spaces $\{M_i\}_{i=1}^m$ and map objects into these projected spaces.

Then, we construct a suitable index structure to characterize data distributions in $\{M_i\}$. Because the projected space is not one-dimensional, we cannot directly use an individual B+-tree to record the complete information of objects in it. To overcome this challenge, in each M_i , we select α reference objects $\{C_j^i\}_{j=1}^\alpha$ and construct β rings $\{R_{j,l}^i\} (1 \leq l \leq \beta)$ sharing each center C_j^i so as to contain all the objects in M_i . In addition, we select an additional reference vector v_i in each M_i . For each ring $R_{j,l}^i$, we translate v_i such that its starting point coincides with C_j^i and compute the reference angle of each object in $R_{j,l}^i$ to v_i . Then, we can construct a B+-tree $T_{j,l}^i$ corresponding to $R_{j,l}^i$ such that the reference angle of each object in $R_{j,l}^i$ is considered as the key of this object in $T_{j,l}^i$. All such B+-trees form the index structure of R2LSH.

In the query phase, given query q , we set up a query-centric ball $B(g_i(q), r)$ in each M_i . Here, the center $g_i(q)$ is the image of q in M_i , and the radius r is initially set to zero. As the search proceeds, we increase r (for each ball) in steps of Δr and scan objects newly included in the expanded balls. Two conditions play important roles in this process. The first is the candidate condition, which determines whether the scanned object can be presently considered as a candidate. Similar to C2LSH [5] and QALSH [8], the candidate condition is obtained by collision testing. The second is the termination condition. We stop the expansion of balls once and return results from candidates if this condition is satisfied. This condition is intimately related to our probability guarantee, which will be introduced in detail in Sec. VI.

In Table I, we summarize a few notations that will be used frequently.

V. INDEXING PHASE

In this section, we introduce a method to construct the index structure of R2LSH (Algorithm 1). The whole indexing phase can be divided into four steps: 1) construction of projected spaces, 2) partition of projected spaces, 3) reference vector selection, and 4) indexing objects by B+-trees. These are introduced below in that order.

A. Construction of projected spaces

First, we generate $2m$ hash functions $\{h_{a_i}\}_{i=1}^{2m}$, where the vectors $\{a_i\}_{i=1}^{2m}$ are i.i.d samples from the Gaussian distribution $\mathcal{N}(0, I_d)$. For any object $o \in \mathbb{R}^d$, we can compute its hash values as $(h_{a_1}(o), h_{a_2}(o), \dots, h_{a_{2m}}(o))$. Furthermore, we couple successive pairs of values as $g_i(o) = (h_{a_{2i-1}}(o), h_{a_{2i}}(o))$ to construct a two-dimensional projected space $M_i \in \mathbb{R}^2$ ($1 \leq i \leq m$). o is mapped to this space by g_i as $g_i(o) \in M_i$.

B. Partition of projected spaces

Let us focus on a projected space M_i . The partition process is divided into two steps: 1) select the necessary number of reference objects in M_i , and 2) construct concentric rings centered at each reference object such that each object in M_i is uniquely contained in one of the rings. We consider the following two strategies for selecting reference objects.

Cluster-based selection. If the dataset $\mathcal{D}$ is well clustered, it is highly likely that the cluster structure is maintained uniform in M_i . Therefore, M_i can be considered as a subspace of the original space. In this case, we determine α clusters in M_i by clustering. Then, the centroid of each cluster is considered as a reference object. By clustering, we can make each object lie in a large ring centered at a certain cluster centroid. Finally, for each centroid, we equi-partition its corresponding ring into β concentric sub-rings $\{R_{j,l}^i\}_{l=1}^\beta$ centered at C_j^i such that each $R_{j,l}^i$ ($1 \leq l \leq \beta$) contains an identical number of objects.

Simple selection. In certain cases, we need to address dynamic datasets with distributions drifting over time. In this case, for convenient update, we select the center of the data space as a reference object ($\alpha = 1$). Then, we partition its corresponding ring into β sub-rings of equal area.

C. Reference Vector Selection

In this section, we describe a method to determine a reference vector v_i in each M_i ($1 \leq i \leq m$). Because objects in a ring are distinguished only by their angles to the reference vectors in B+-trees, the angle between any two v_i s is expected to be close to $\pi/2$ to the extent feasible. First, note that two axes of M_i are constructed by randomly generated vectors a_{2i-1} and a_{2i} . These can be represented as $\sum_{j=1}^n a_j^i e_j$ and $\sum_{j=1}^n b_j^i e_j$, respectively. Here, $\{e_j\}_{j=1}^d$ is the natural basis of the original d -dimensional space, and $\{a_j^i\}, \{b_j^i\}$ are coefficients. Thus, v_i can be represented as $v_i(\lambda_1^i, \lambda_2^i) = \sum_{j=1}^m (\lambda_1^i a_j^i + \lambda_2^i b_j^i) e_j$, where λ_1^i and λ_2^i are coefficients to be determined.

Then, in the average sense, we adopt the following strategy to determine $\{v_i\}$ sequentially in the greedy way.

Step 1: For the first projected space M_1 , we arbitrarily select a normalized vector in M_1 as v_1 .

Step 2: Suppose that we have determined $i-1$ reference vectors $v_1, \dots, v_{i-1}$. For v_i , we require $\sum_{j=1}^{i-1} \langle v_i(\lambda_1^i, \lambda_2^i), v_j \rangle = 0$. In addition, we require $(\lambda_1^i)^2 + (\lambda_2^i)^2 = 1$ and $\lambda_1^i \geq 0$ for the norm of v_i to have no effect on the angle computation. It is evident that under the above constraints, λ_1^i and λ_2^i can be obtained uniquely, and thereby, v_i can be determined. Then, we repeat Step 2 until all the v_i s are obtained.

D. Indexing objects by B+-trees

In Figure 3, we describe a method to construct B+-trees in a projected space, say M_i (others are addressed similarly). First, note that after the preceding steps, each mapped object is included in only one ring of the ring set $\{R_{j,l}\}$ in M_i , e.g., $o_1, o_2 \in R_{1,2}$ and $o_3 \in R_{2,1}$. For each ring $R_{j,l}$, we construct a B+-tree $T_{j,l}$ to index all the objects in $R_{j,l}$. Let us focus on the ring $R_{1,2}$ and the corresponding vector v centered at C_1 . We measure the angle of each object in $R_{1,2}$ to v as its key. Thus, the key of o_1 is $\pi/3$ and that of o_2 is $4\pi/3$. We can construct a B+-tree $T_{1,2}$ with these keys. Note that in this manner, each object belongs to only one of the B+-trees.

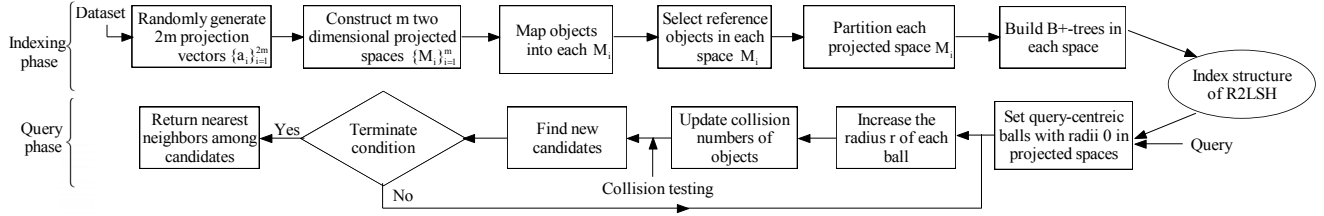


Figure 2: Overview of R2LSH

VI. QUERY PHASE

In this section, we illustrate the query phase of R2LSH. The goal of this phase is to identify candidates by query-centric balls. In Sec. VI-A, by deriving certain fundamental relationships, we demonstrate a method to determine the radii of rings to ensure that o^* , the nearest neighbor of o , is identified with the expected success probability. In Sec. VI-B, we verify that all the objects inside the query-centric balls are necessarily scanned by our index structure. After the descriptions of these preparations, we present the algorithm and the related probability guarantee, in Secs. VI-C and VI-D.

A. Fundamental relationships

Algorithm 1: Indexing phase of R2LSH

Input: $\mathcal{D}$ is the dataset; m is the number of projected spaces; α is the number of reference objects in a projection space; β is the number of rings centered at each reference object;

Output: the index structure $\{R_{j,l}^i, T_{j,l}^i\}$

- 1 Generate $2m$ hash functions $\{h_i\}_{i=1}^{2m}$ by random projection;
- 2 Construct m two-dimensional projected spaces $\{M_i\}_{i=1}^m$;
- 3 **for** $i = 1$ **to** m **do**
- 4 $\forall o \in \mathcal{D}$ map o to M_i by $o \rightarrow (h_{2i-1}(o), h_{2i}(o))$;
- 5 Select α reference objects $\{C_j^i\}_{j=1}^\alpha$ in M_i ;
- 6 Select reference vector v_i ;
- 7 **for** $j = 1$ **to** α **do**
- 8 Construct β concentric rings $\{R_{j,l}^i\}_{l=1}^\beta$ centered at C_j^i ;
- 9 **for** $l = 1$ **to** β **do**
- 10 $\forall o \in \mathcal{D}$ compute o 's reference angle to v_i ;
- 11 Construct B+-tree $T_{j,l}^i$;
- 12 **Return** $\{R_{j,l}^i, T_{j,l}^i\}$ ($1 \leq i \leq m, 1 \leq j \leq \alpha, 1 \leq l \leq \beta$);

First, let us consider a simpler problem. Given query q and error rate $0 < \delta < 1$, when $s^* = \|o^* - q\|$ is known before the search, how do we identify o^* with probability at least $1 - \delta$? According to (2), we have the following equation:

$$\Pr[\Delta_i(o^*) \leq r] = \Phi\left(\frac{r^2}{(s^*)^2}\right), \quad (3)$$

where r is a positive number, i ($1 \leq i \leq m$) is an arbitrary suffix of the projected space, and Φ is the CDF of the chi-

square distribution $\chi^2(2)$. Then, according to the definition of $\Delta_i(o)$, (3) is re-expressed as

$$\Pr[g_i(o) \in B(g_i(q), r)] = \Phi(r^2/(s^*)^2). \quad (4)$$

Meanwhile, from the fact that $\{M_i\}_{i=1}^m$ are constructed independently, we know that $\#Col(o^*; r)$ (the number of those projected spaces where o^* falls inside the query-centric balls with radii r) follows the Bernoulli distribution $Ber(m, \Phi(r^2/(s^*)^2))$. Therefore, by the Hoeffding's inequality, we have

$$\Pr[\#Col(o^*; r) \geq \tau] \geq 1 - \exp\{-2m(\Phi(r^2/(s^*)^2) - \tau/m)^2\} \quad (5)$$

where the positive integer τ denotes a threshold. This will be determined subsequently. For a specified error rate δ , to permit $\Pr[\#Col(o^*; r) \geq \tau] \geq 1 - \delta$ to hold, we only require that the following inequality holds:

$$\Phi\left(\frac{r^2}{(s^*)^2}\right) \geq \sqrt{\frac{1}{2m} \log \frac{1}{\delta}} + \frac{\tau}{m} = \lambda(m, \delta, \tau) \quad (6)$$

When m , δ , and τ are evident from the context, we use λ for $\lambda(m, \delta, \tau)$. Because $\Phi(x)$ is monotonically increasing in x and a bijection from R^+ to $[0, 1]$, it suffices to adopt an r such that

$$r \geq r^* = \sqrt{(s^*)^2 \Phi^{-1}(\lambda)}. \quad (7)$$

Under (7), we can guarantee that o^* passes the collision testing, i.e. $\#Col(o^*; r) \geq \tau$, with probability at least $1 - \delta$. Apparently, all the objects that pass this collision testing are considered as candidates. As the input parameters λ , m , and δ are specified, the collision threshold τ is determined from (6) by

$$\tau = \left\lceil m\lambda - \sqrt{\frac{m}{2} \ln \frac{1}{\delta}} \right\rceil. \quad (8)$$

However, in practice, we cannot directly determine r^* by (7) because s^* is unknown. However, this challenge can be overcome by the dynamic counting technique proposed in [5]. At the beginning of the search, we set a query-centric ball of radius zero in each M_i . As the search proceeds, we increase the radius r of each ball in steps of Δr and update the collision

numbers of data objects. During this process, we keep the nearest neighbor $\hat{o}$ of q among the accessed candidates and accurately calculate its distance to q , i.e., $\hat{s} = \|\hat{o} - q\|$. If we identify that $r \geq \hat{s}\sqrt{\Phi^{-1}(\lambda)}$ holds for a fixed λ , then we know $r \geq r^*$ from (7). This is because

$$r \geq \hat{s}\sqrt{\Phi^{-1}(\lambda)} \Rightarrow r \geq s^*\sqrt{\Phi^{-1}(\lambda)} \Rightarrow r \geq r^*. \quad (9)$$

Thus, $r > \hat{s}\sqrt{\Phi^{-1}(\lambda)}$ is considered as the termination condition of R2LSH.

B. Scanning Process

The remaining problem is to identify the objects that lie in query-centric balls. Because all the objects are contained in one of the rings in each projected space, we need to consider only the relationship between the query-centric ball and a ring. Given a query ball $B(g_i(q), r)$ and a ring region R centered at C with inner radius $\underline{r}$ and outer radius $\bar{r}$ in M_i . Let $s_0 = \|C - g_i(q)\|$, we consider the following two cases:

Case 1: $s_0 - r > \bar{r}$ or $s_0 + r < \underline{r}$. Evidently, in this case, the ball does not intersect with R . Therefore, we omit the examination of R .

Case 2: $\underline{r} - r \leq s_0 \leq r + \bar{r}$. In this case, we first compute the reference angle θ_q of $g_i(q)$ to the reference vector v_i of R . Then, we consider three more detailed cases and compute θ as follows:

$$\theta = \begin{cases} \arccos\left(\frac{(s_0)^2 + \bar{r}^2 - r^2}{2s_0\bar{r}}\right) & (s_0)^2 - r^2 > \bar{r}^2 \\ \arcsin(r/s_0) & \underline{r}^2 \leq (s_0)^2 - r^2 \leq \bar{r}^2 \\ \arccos\left(\frac{(s_0)^2 + \underline{r}^2 - r^2}{2s_0\underline{r}}\right) & (s_0)^2 - r^2 < \underline{r}^2 \end{cases} \quad (10)$$

It is evident from elementary calculation that given $r > 0$, we only need to scan those objects in R whose reference angles to v_i lie in $[\theta_q - \theta, \theta_q + \theta]$ so that all the objects that fall in $B(g_i(q), r)$ can be covered. It is convenient to access these objects in the external memory because they are sequentially stored (in mod 2π) in the leaf nodes of T , i.e., the corresponding B+-tree of R . Therefore, the complete scanning process of R can be illustrated as follows (Figure 3): given a query q , we first determine the location of θ_q in the leaf nodes of T . In each step, according to r , we can determine the search interval $[\theta_q - \theta, \theta_q + \theta]$ in which the objects need to be scanned. In the next step, r is increased by Δr , and θ is updated according to (10). At this stage, we only need to extend $[\theta_q - \theta, \theta_q + \theta]$ in two directions. This can be realized conveniently at leaf levels of B+-trees. If $\theta_q - \theta < 0$ or $\theta_q + \theta > 2\pi$, we only need to replace $\theta_q - \theta$ by $\theta_q - \theta + 2\pi$ or replace $\theta_q + \theta$ by $\theta_q + \theta - 2\pi$. This implies that the interval extends from the other end of the leaf level.

It is notable that for a few objects that lie outside balls, they are also likely to lie in the interval that needs to be examined (such as o_4 in Figure 3). However, for the desired object o^* , this only marginally increases the likelihood that it passes the

collision testing. Therefore, the probability guarantee obtained in Sec. VI-A still holds.

Finally, the setting of Δr can be computed by a minimum heap. Specifically, we only scan a leaf node of a B+-tree in each step. According to the key value θ of the last scanned object in this step and (10), we can reversely compute r . Thus, for each B+-tree that needs to be examined, we can maintain two corresponding values r_0 and r_1 . Here, r_0 and r_1 correspond to the left and right directions, respectively. By means of the minimum-heap, in each step, we can determine the B+-tree and the search direction corresponding to the minimum r_0 or r_1 , and examine the next leaf node.

C. Algorithm and Quality guarantee

The query phase is displayed as Algorithm 2 for solving the c -ANN problem ($c \geq 1, k = 1$). The probability guarantee of Algorithm 2 is as follows:

Algorithm 2: Query phase of R2LSH

Input: $\mathcal{D}$ is the dataset; q is the query point; m is the number of projected spaces; $\{R_j^i, T_j^i\}$ ($1 \leq i \leq m, 1 \leq j \leq N$) is the index structure, where N is the number of rings in a projected space; δ ($0 < \delta < 1$) is the error rate; λ ($0 < \lambda < 1$) is the parameter for computing the threshold; c ($c \geq 1$) is the approximation ratio;

Output: $o_{\min}$

```

1  $r \leftarrow 0, \hat{s} \leftarrow \infty, o_{\min} \leftarrow NULL;$ 
2  $\tau \leftarrow \lfloor m\lambda - \sqrt{m \ln(1/\delta)/2} \rfloor;$ 
3 while  $r \leq \sqrt{\Phi^{-1}(\lambda)}\hat{s}/c$  do
4    $r \leftarrow r + \Delta r$  ( $\Delta r > 0$ );
5   for  $i = 1$  to  $m$  do
6     for  $j = 1$  to  $N$  do
7       if  $B(g_i(q), r) \cap R_j^i \neq \emptyset$  then
8         Update the search interval in  $T_j^i$  by (10);
9       else
10        continue;
11   $\forall o \in \mathcal{D}$  update  $\#Col(o; r)$  when necessary;
12  if  $o$  is not visited and  $\#Col(o; r) \geq \tau$  then
13    Calculate  $\hat{s} = \|q - o\|;$ 
14    Update  $o_{\min}$  and  $\hat{s}$  when necessary;
15 Return  $o_{\min}$ 
```

Theorem 1. For a specific query q and approximation ratio ($c \geq 1$), Algorithm 2 returns a c -approximate NN of q with probability at least $1 - \delta$.

Proof. Let us consider the following two cases in order:

Case 1: ($c = 1$). According to the termination condition, the query phase can terminate only when $r \geq \sqrt{\Phi^{-1}(\lambda)}\hat{s}$ holds. By the preceding analysis, o^* is returned with probability at least $1 - \delta$.

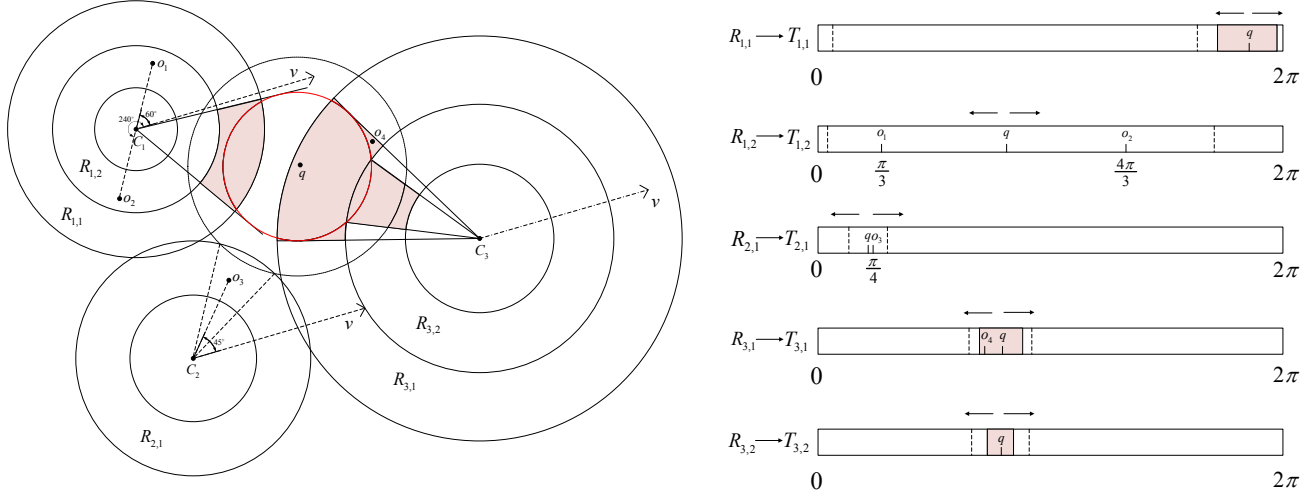


Figure 3: Example of B+-trees for three clusters in a projected space (Left: geometrical relationship. Right: leaf levels of B+-trees). Three sets of co-centered rings are constructed such that each set includes part of the projected objects in ring. A ring set $\{R_{j,l}\}$ centered at C_j constructs a B+-tree set $\{T_{j,l}\}$. A B+-tree $T_{j,l}$ is constructed over the projected objects in $R_{j,l}$ with respect to their angles to a reference vector v . For example, $o_1 \in R_{1,2}$ is included only in $T_{1,2}$, and the location is $\pi/3$ when a query is mapped into this space as q . A query-centric circle of specified radius r (a red circle) is simulated by those B+-trees as follows. The angle of q to v is examined, and each $T_{j,l}$ is searched. Among these, only the objects in the search interval of q need to be examined (for counting). Presently, o_4 is identified. More objects can be identified by expanding the query-centric ball (the dotted circle) and increasing the size of the search intervals (dotted rectangles).

Case 2: ($c > 1$). First, three events need to be defined. E_0 : $\hat{o}$ is a c -approximate object, E_1 : $\#Col(o^*; r^*) \geq \tau$, and E_2 : $r < r^*$. We need to establish that $\Pr[E_0] \geq 1 - \delta$. When E_2 occurs, we obtain $\hat{s} \leq cs^*$ according to the definition of E_2 . This implies that we have identified c -approximate objects, i.e., E_0 occurs. When both $\bar{E}_2$ and E_1 occur, we also know E_0 occurs for o^* is found. Because E_2 and $E_1 \cap \bar{E}_2$ are disjoint, we obtain the following inequalities:

$$\Pr[E_0] \geq \Pr[E_2] + \Pr[E_1 \bar{E}_2] \quad (11)$$

$$\geq \Pr[E_1 E_2] + \Pr[E_1 \bar{E}_2] \quad (12)$$

$$= \Pr[E_1] \quad (13)$$

Because $\Pr[E_1]$ has been established to be at least $1 - \delta$ in Case 1, we conclude. $\square$

D. Extension to c - k -ANN search

Algorithm 2 can be modified conveniently as follows to support c - k -ANN search:

- (i) A priority queue Q needs to be maintained to store k nearest neighbors $\hat{o}_1, \dots, \hat{o}_k$ among the candidates.
- (ii) The termination condition in Step 3 needs to be modified as $r \leq \sqrt{\Phi^{-1}(\lambda)} \hat{s}_k / c$. Here, $\hat{s}_k = \|\hat{o}_k - q\|$.

When we apply our previous analysis on o^* to o_i^* ($1 \leq i \leq k$) (where o_i^* is the true i th nearest neighbor of q), in accordance with Theorem 1, we have the following corollary:

Corollary 1. For an arbitrary suffix i ($1 \leq i \leq k$), the probability that a c -approximate nearest neighbor of o_i^* is returned by Algorithm 2 is at least $1 - \delta$. Moreover, if c is set to one, the expectation of recall rate is at least $1 - \delta$.

VII. DISCUSSION

A. Complexity Analysis

1) **Time Complexity:** In the indexing phase, R2LSH consumes $O(nd)$ time to compute the images of each object in the projected spaces. Then, R2LSH consumes $O(n)$ time to partition the spaces based on reference objects and to select reference vectors. For each ring, R2LSH consumes $O(n)$ time to compute the reference angle of each object to the reference vector and $O(n \log n)$ time to construct the corresponding B+-tree. Therefore, the time complexity of the indexing phase is $O(nd + n \log n)$.

In the query phase, R2LSH requires $O(n)$ time to scan all the B+-trees and $O(nd)$ time to accurately compute the distances between q and the candidates. This implies that in the worst case, the time complexity of R2LSH is linear. This does not imply that R2LSH is inefficient because unlike many other c -ANN methods, R2LSH can support any value of c (≥ 1), with probability guarantee. To the authors' knowledge, there is no search method can achieve the same goal in sub-linear complexity. On real datasets, R2LSH only computes the exact distances of q to a very small number of objects by the tuning of c and thus performs highly effectively, as demonstrated by experiments described in Sec. VIII.

2) *Space Complexity*: In the indexing phase, R2LSH first consumes $O(md)$ space to restore m projection vectors. Then, R2LSH consumes the corresponding B+-tree for each ring, consuming $O(n)$ space to restore the projections of the objects. During the phase of constructing B+-trees, R2LSH requires a buffer of size equal to the I/O page size, to restore a leaf/index node of the B+-tree. Therefore, the total space cost is $O(n + md)$.

In the query phase, before the search, R2LSH first needs to load certain auxiliary information of the index structure into the main memory. The information includes those on (1) $2m$ projection vectors, (2) reference objects and the reference vector in each M_i , and (3) the inner and outer radii of each ring, which is totally $O(md)$. During the search process, R2LSH needs to maintain an array of size n to record the present collision numbers of the objects. In addition, for each ring that needs to be searched, R2LSH maintains two buffers to restore the I/O pages read from disks. This is because the leaf level is searched in two directions. If the I/O page size is set to 4KB, the space cost of this part is at most $8m\alpha\beta$ KB. α and β are defined in Table I. Because $n \gg m\alpha\beta$ for large datasets, the total space cost in the query phase is also $O(n + md)$.

B. Handling Update and parameter setting

Update. For the datasets updated frequently, we recommend users to adopt a simple selection strategy. This is because the learning step in the indexing phase is not required, implying that the cost of re-learning owing to the alterations in data distributions could be saved. In the experiments, we demonstrate that for most datasets, a simple selection strategy can be effective. Meanwhile, because our index structure is formed by B+-trees, the update (insertion, deletion, and modification) can be supported conveniently.

Parameter setting. The parameters that need to be determined are m , λ , and β . It is evident that their optimal values depend on both data distribution and query distribution. To support dynamic and large datasets, in this study, we experimentally determined the default values of these three parameters and applied these values to all the real datasets. The experiments revealed that R2LSH could perform effectively under such default setting of parameters.

VIII. EXPERIMENTS

R2LSH was implemented in C++. All the experiments were carried on a PC with Intel(R), 3.40 GHz i7-4770 eight-core processor with 8 GB RAM, in Ubuntu 16.04, unless stated otherwise.

A. Experiment Setup

1) Benchmark Methods:

- **SRS [15].** We select SRS-2, a variant of SRS, in our experiments. This is because SRS-2 supports arbitrary $c \geq 1$ and $0 < \delta < 1$. The dimension of the R-tree was set to eight, as recommended in [7], [12], [15]. δ was set to 0.1.

- **QALSH [8].** According to the authors' recommendations, δ was set to $1/2 + 1/e$. The number of hash functions m and other internal parameters were computed according to the methods proposed in [8] as well.
- **R2LSH.** δ was set to 0.1. The parameters λ , m , and β were set as described in Sec. VIII-A4. In addition, to compare R2LSH and the other LSH methods without bias, we adopted the simple selection strategy ($\alpha = 1$) introduced in Sec. V-B (unless stated otherwise). This causes all LSH methods to be learning-free.
- **HD-index [1].** We select HD-index as a representative of non-LSH methods (without quality guarantees). All the internal parameters of HD-index were adjusted to be experimentally optimal, as recommended in [1].
- **PQBF [13].** PQBF is a representative of learning methods. All of the internal parameters were set as suggested by its authors.

Since all methods expect for PQBF are learning-free methods, we focus on their comparison before Sec. VIII-G. The comparison between R2LSH and PQBF will be shown in Sec. VIII-G.

2) *Datasets*: We select ten real datasets of different sizes [1], [12], [15] (Table II):

Table II: Real datasets

Dataset	Dimension	Size	Type
Sun	512	79,106	Image
Enron	1369	94,987	Text
Msong	420	992,272	Audio
Deep	256	1,000,000	Image
Gist	960	1,000,000	Image
Imagenet	150	2,340,373	Image
Tiny10M	384	10,000,000	Image
Sift10M	128	11,164,666	Image
Tiny80M	384	79,302,017	Image
Sift1B	128	999,494,170	Image

The I/O page sizes, denoted by B , were set to 4KB on all the datasets, as suggested in [1], [13], etc.

3) *Evaluation metrics*: We used the following metrics for the performance evaluation.

- **I/O cost.** I/O cost, which denotes the number of pages to be accessed, is an important metric for external memory algorithms. We follow SRS and QALSH to use this metric to evaluate the efficiency. As discussed earlier, I/O costs consist of the overhead for both index and data access.
- **Running time.** The running time for processing a query is also considered. It is defined as the wall clock time for a method to solve the k -ANN problem.
- **Overall ratio.** For methods to solve c - k -ANN search, overall ratio is a standard metric used to measure the query accuracy. For c - k -ANN search, the overall ratio is defined as $\frac{1}{k} \sum_{i=1}^k \|q - \hat{o}_i\| / \|q - o_i^*\|$.
- **Recall.** Recall is used as another important metric to measure the accuracy of algorithms. Its value is equal to the

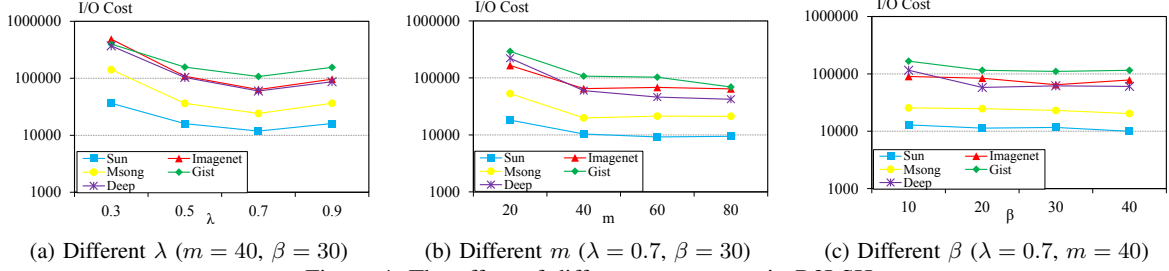


Figure 4: The effect of different parameters in R2LSH

ratio of the number of returned true nearest neighbors to k (for k -ANN search).

4) *Parameter settings of R2LSH*: We recommend users to set λ , m , and β to 0.7, 40, and 30, respectively, by default. This is because such settings are suitable for most datasets. In Figure 4, we demonstrate the effect of different parameters on I/O costs, through experiments on five datasets.

- Figure 4(a) shows that $\lambda = 0.7$ is suitable because the least I/O cost is incurred on each dataset under this setting. If λ is set too small or too large, it will be challenging to distinguish NNs from other false objects owing to their close possibilities of falling inside query-centric balls.
- Figure 4(b) shows that m should not be too small because R2LSH requires an adequate number of balls for the collision testing to be efficient. Meanwhile, when $m \geq 40$, almost all the curves remain stable except the curve of Gist. This implies that it is effective to set m to 40. With regard to Gist, we can achieve higher performance when m is set larger because Gist is a high-dimensional dataset and requires more balls in the indexing phase.
- Figure 4(c) shows that the I/O cost required under $\beta \geq 20$ is marginally less than that under $\beta = 10$. This is because we can scan a lower number of objects by using more rings. Meanwhile, β should not be too large because we have to generate a B+-tree for each ring. Therefore, we set β to 30. This is observed to be suitable for all the five datasets.

5) *Cluster-based selection vs Simple selection*: On all the datasets of size within 1 M, we compared R2LSH with $\alpha = 4$ (four reference objects were determined by K-means) and the standard R2LSH ($\alpha = 1$) with the simple selection strategy, under $c = 1$. Compared with the standard R2LSH, R2LSH ($\alpha = 4$) can save 5% ~ 20% I/O cost on each dataset. This implies that R2LSH based on cluster-based selection performs marginally higher than the standard R2LSH even in the general case.

B. Efficiency of R2LSH

In this section, we describe the experiment by which it was demonstrated that the design of R2LSH is reasonable. It concerns the following two aspects: (1) the superiority of B+-trees over other tree structures, and (2) the superiority of Dimension 2 (the subspace dimension of R2LSH) over other dimensions. All the experiments described in this section were performed on Tiny10M and Sift10M.

1) *B+-tree vs. R-tree*: We first demonstrate that B+-trees are superior to other tree structures in the searching process of R2LSH. Here, we select R-tree as the benchmark tree structure because it can be realized in the external memory. In this case, we adjust the number of subspaces and the dimension of R-trees for them to be experimentally optimal. Moreover, we use an R-tree to scan objects sequentially as the radius of the search ball increases in each subspace. The comparison results are presented in Table III. The results reveal that R-trees are less effective than B+-trees in our searching scheme. This is because objects are stored sequentially on B+-trees, whereas the scanning on R-trees results in significantly more random I/Os (other low-dimensional tree-structures also exhibit this limitation). Thus, for our algorithm to save time, we use B+-trees to form our data structure.

Table III: B+-tree vs. R-tree on running times(s) necessary to achieve overall ratio 1.01. @k means the number of nearest neighbors.

	Sift(@1)	Sift(@10)	Tiny(@1)	Tiny(@10)
B+-tree	5.3	7.5	9.9	11.2
R-tree	23.2	33.3	46.3	60.7

Table IV: Ratios of I/O costs in index access of R2LSH to those of QALSH. @k means the number of nearest neighbors.

	Sift(@1)	Sift(@10)	Tiny(@1)	Tiny(@10)
Ratio	0.05	0.04	0.06	0.06

2) *Dimension 2 vs. Dimension 1*: Because the query-centric hash bucket can be considered as the one-dimensional query-centric ball, it suffices to demonstrate the superiority of R2LSH (Dim 2) over QALSH (dim 1). In this section, we focus only on the comparison results of I/O costs in index access, which reflects the most important advantage of R2LSH over QALSH. The results in Table IV illustrate that R2LSH requires 4%–6% of the I/O cost for index access of QALSH because of the higher selectivity of two-dimensional query-centric balls.

3) *Dimension 2 vs. higher dimensions*: R2LSH can be conveniently generalized to support higher subspace dimensions. Here, for each hyper-ring, we construct a B+-tree to store the angles of objects to a reference vector (although we can construct multiple B+-trees for each subspace, this will cause the total number of B+-trees to be excessive) and

search for the candidates similarly. Table V presents the results under different subspace dimensions. The results reveal that when we select a higher dimension (> 2), the performance is inferior to that under Dimension 2. This is because it is more challenging to prune false objects in subspaces of higher dimensions unless more B+-trees or other low-dimensional tree structures are used. However, as discussed earlier, this will also incur significant additional cost.

Table V: Running times(s) necessary to achieve overall ratio 1.01 under different subspace dimensions. @k means the number of nearest neighbors.

	Sift(@1)	Sift(@10)	Tiny(@1)	Tiny(@10)
Dim 2	5.3	7.5	9.9	11.2
Dim 3	13.2	18.5	26.9	30.8
Dim 4	15.4	22.7	32.3	35.6

C. Index Size, Indexing Time, and Memory Consumption

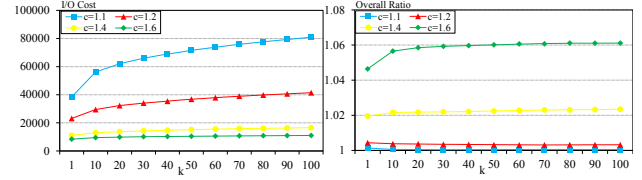
Table VI: Index sizes ($c = 2$). CR denotes crash in the indexing phase.

Dataset	SRS	QALSH	R2LSH	HD-index
Sun	4.1MB	21.2MB	13.5MB	250MB
Enron	4.6MB	26.5MB	16.6MB	1.5GB
Msong	47.5MB	341.1MB	164.4MB	3.9GB
Deep	47.4MB	350.9MB	169.1MB	2.0GB
Gist	47.3MB	350.9MB	169.1MB	18.1GB
Imagenet	110.8MB	836.0MB	380.0MB	4.1GB
Tiny10M	476.7MB	3.8GB	1.6GB	31.3GB
Sift10M	524.7MB	4.1GB	1.8GB	10.2GB
Tiny80M	3.6GB	CR	12.6GB	241.3GB
Sift1B	51.2GB	CR	177.8GB	1.2TB

Table VI presents the index sizes of three methods. Because the index size of QALSH depends on the approximation ratio c , we report only the results under $c = 2$. According to the results, on each dataset, the index size of SRS is the smallest, followed by R2LSH, QALSH, and HD-index. This is because the index structure of SRS is an R-tree, whereas R2LSH and QALSH use multiple B+-trees. Meanwhile, although the number of B+-trees of R2LSH (1200) is more than that of QALSH (60-100), each object appears only 40 (equal to the number of projected spaces) times in the index structure of R2LSH. Therefore, the index size of R2LSH can be less than that of QALSH.

With regard to the indexing time, on our PC, R2LSH is the fastest, followed by QALSH, SRS, and HD-index, on all the datasets. For example, on Sift1B, R2LSH, SRS, and HD-index consume 13 h, 5 days, and 11 days, respectively, to construct the index. The construction of B+-trees is more time-consuming than that of an R+-tree. Thereby, SRS consumes more time in the indexing phase although its index size is the smallest among the three LSH-methods.

In addition, we report the main memory consumption of each method on the largest dataset Sift1B as follows. In the



(a) Gist, I/O Cost (b) Gist, Overall Ratio
Figure 5: I/O Cost and accuracy of R2LSH vs. c

indexing phase, SRS, R2LSH, and HD-index consume 1.1GB, 1.6GB, and 97MB. SRS and R2LSH are both projection-based methods and require more space in the main memory to store projection values.

D. The effect of c

In Figure 5, we study the effect of different values of the approximation ratio c on the performance of R2LSH. Owing to the space limitation, we present the results only on Gist. Similar trends were observed on other datasets. The following observations are based on the results:

- The real overall ratio is significantly smaller than the value of c , a required lower bound. This is because the probability guarantee of R2LSH is obtained in the worst case, and the performances of R2LSH on real datasets can be significantly higher.
- As c decreases, R2LSH achieves higher query quality at the expense of higher I/O cost. This is consistent with our termination condition in Algorithm 2. This implies that according to the practical requirement, we can achieve a good balance between query cost and precision by tuning the approximation ratio. Figure 5 shows that R2LSH performs well under $c = 1.2$.

E. R2LSH vs. other LSH-based methods

This section describes our comparison of R2LSH with other state-of-the-art LSH-based methods, i.e., SRS and QALSH.

Table VII: I/O costs ($\times 10^5$) necessary to achieve overall ratio 1.01 on Tiny80M and Sift1B. @k means the number of nearest neighbors.

	Sift(@10)	Sift(@100)	Tiny(@10)	Tiny(@100)
R2LSH	81	98	6	9
SRS	307	379	55	70

1) *Comparison on I/O costs*: First, we compare the required I/O costs of R2LSH, QALSH, and SRS (Figure 6 and Table VII). Please note that we measured I/O costs or time necessary for achieving a specified value of overall ratio instead of a same value of approximation ratio c ¹. This is because R2LSH, QALSH, and SRS adopt different methods to obtain probability guarantees. Consequently, SRS incurs less I/O cost, whereas QALSH achieves higher accuracy for the same value

¹In previous papers, $c = 2$ is considered as the standard setting. However, it is impractical because for many high-dimensional real datasets, over 30% of the data objects are candidates when $c = 2$.

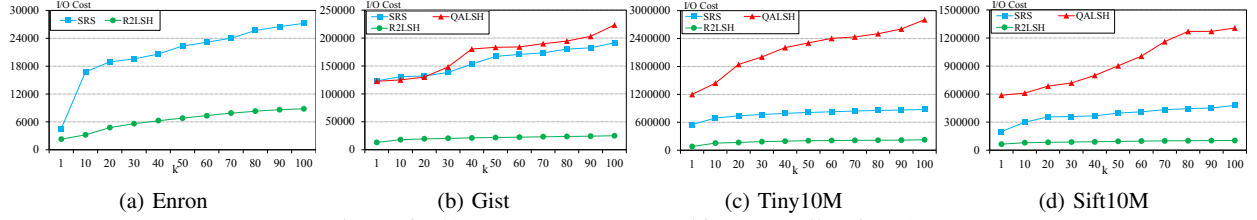


Figure 6: I/O costs necessary to achieve overall ratio 1.01

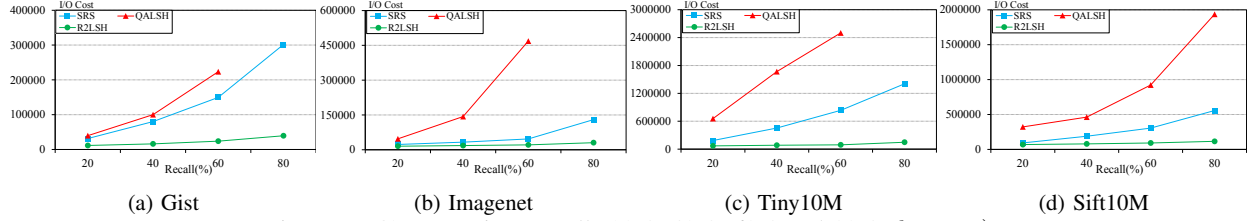


Figure 7: I/O costs given recalls 20%, 40%, 60% and 80% ($k = 100$)

of c . Note that all the three methods can support arbitrary c ($c > 1$) and can achieve different accuracies by tuning c . Therefore, for fair comparison, we set the overall ratio to a constant and measure the I/O cost they incur to attain the objective overall ratio by tuning c . In Figure 6, we fix the overall ratio to 1.01 and compare the performances of the three methods. $c = 1.01$ is selected because it is adequately low, and all the methods can achieve this ratio on most of the datasets. The results reveal that the performance of R2LSH is the highest on each dataset. Specifically, R2LSH incurs 15%–30% of the I/O costs incurred by SRS or QALSH to achieve an identical precision level. This reveals that R2LSH is more I/O efficient than other LSH-based methods. We also observe that the performance of QALSH degrades significantly as the data size increases. This is because QALSH first fixes the size of the candidate set and then computes the number of required hash functions. For large datasets, a similar setting makes the candidate set of QALSH too small whereas the I/O cost of index access becomes excessively large. This makes the search inefficient.

In Figure 7, we vary the targeted recall and compare these methods under $k = 100$. We can see that under different precision levels, R2LSH always exhibits the highest performance. In addition, as the targeted recall increases, the advantage of R2LSH over SRS or QALSH becomes more obvious.

2) *comparison on running times*: We also compare the running times of the three methods in Table VIII. We first observe that R2LSH exhibits apparent advantages over SRS on speeds. This is because the index structure of R2LSH is formed by B+-trees. During the scanning process, the objects on B+-trees can be scanned sequentially, which is efficient in terms of time-consumption. In contrast, SRS processes objects by an R-tree, which incurs more time for distance computation and searching for new index nodes.

Meanwhile, we observe that R2LSH runs significantly faster than QALSH, particularly on large datasets. This is consistent

Table VIII: Speeds(s) given recall 70% ($k = 100$). 70% is selected because it can be achieved by three methods and corresponds to targeted overall ratio of approximately 1.01.

Dataset	R2LSH	Gain of R2LSH over	
		QALSH	SRS
Sun	0.28	1.00x	3.02x
Enron	0.41	\	2.97x
Msong	0.93	2.83x	4.28x
Deep	1.41	4.33x	5.63x
Gist	2.70	5.66x	3.70x
Imagenet	2.53	12.19x	5.72x
Tiny10M	12.66	14.61x	6.77x
Sift10M	9.51	9.15x	7.13x
Tiny80M	81.53	\	7.82x
Sift1B	793.18	\	7.27x

with the results on I/O costs. It is noteworthy that on the small dataset Sun, the running times of two methods are very close. This is because R2LSH constructs more B+-trees than QALSH on small datasets, which incurs more time for locating projections of the query. However, such cost is comparatively small for large datasets.

F. R2LSH vs. HD-index

In this section, we compare R2LSH with HD-index, a state-of-the-art non-LSH method. Because HD-index could not solve the c - k -ANN problem, we use only the recall to measure its query accuracy. In addition, we use running times to compare the speeds of two methods. The comparison results are listed in Table IX.

First, we observe that R2LSH displays a significantly higher recall rate than HD-index on each dataset. More specifically, we observe that (1) the performances of HD-index on high-dimensional or hard datasets(Gist, Tiny) degrade significantly; and (2) the recall of HD-index decreases as the size of the dataset increases(Tiny, Sift), whereas that of R2LSH remains

Table IX: R2LSH($c = 1.2$) vs. HD-index, $k = 100$

Dataset	Recall(%)		Running time(s)	
	HD-index	R2LSH	HD-index	R2LSH
Sun	53	84	0.56	0.37
Enron	68	89	0.92	0.71
Msong	51	89	1.84	1.41
Deep	55	83	1.91	1.79
Gist	20	83	8.4	4.79
Imagenet	35	86	2.21	3.31
Tiny10M	10	88	15.24	27.21
Sift10M	21	87	5.42	13.56
Tiny80M	7	86	50.13	141.54
Sift1B	13	89	102.48	987.85

stable on all the datasets (approximately 85%) because of the rigorous probability guarantees. Note that as shown in the experiments of [1], the query accuracy of HD-index cannot be improved demonstrably by adjusting parameters because they have been experimentally optimal. Thus, R2LSH is a more potential method than HD-index in terms of precision and stability.

R2LSH runs faster than HD-index on datasets of size 1M or less. On datasets of size over 10M, HD-index consumes less time because HD-index examines only a small number of objects on each RDB-tree even for very large datasets. However, this does not imply that R2LSH is inferior to HD-index with respect to time consumption. R2LSH can be accelerated significantly by increasing the c suitably. Consider Tiny80M and Sift1B as examples. When we set c to 1.4, the recalls of R2LSH can still attain over 50% on these two datasets, and the running times are reduced by 65%. This implies that R2LSH could achieve significantly higher recall rates than HD-index with comparative running times on large datasets.

G. R2LSH vs. PQBF

Since PQBF does not scale to 8GB RAM in the indexing phase, its training and indexing were done on another PC with Xeon E5-165, 3.60 GHz six-core processor with 128 GB RAM. Then, we conducted its search on the original PC. The comparison results are shown in Table X. We can see that R2LSH achieves much higher recall than PQBF on each dataset, especially on the hard dataset Tiny. This is because PQBF only checks a few partitions of data. For high-dimensional or hard datasets, such strategy may lead to inaccurate query results due to the difficulty in knowing which partitions are really promising. In addition, it is hard to improve the accuracy of PQBF to a user-specified level due to technical reasons and the lack of theoretical guarantees. By contrast, R2LSH owns strict probability guarantee and takes all objects into consideration such that the query accuracy can always be guaranteed.

With regard to the efficiency, we compared the speeds of two methods. Not surprisingly, PQBF is more efficient than

R2LSH. However, R2LSH is still better in accuracy, stability and supporting dynamic datasets.

Table X: R2LSH($c = 1.2$) vs. PQBF, $k = 100$, speedup denotes the ratio of search time of R2LSH to that of PQBF.

	Gist	Deep	Tiny80M	Sift1B
Recall of R2LSH(%)	83	83	86	89
Recall of PQBF(%)	21	37	5	17
Speedup	12.1	5.2	25.1	22.5

IX. CONCLUSION

In this paper, we propose a novel approximate NN search algorithm called R2LSH. R2LSH works with an arbitrarily small approximation ratio $c \geq 1$ and is guaranteed to identify c - k -ANN with the specified success probability. Compared with available methods, R2LSH saves significant I/O cost and running time under identical answer quality.

REFERENCES

- [1] A. Arora, S. Sinha, P. Kumar, and A. Bhattacharya. Hd-index: Pushing the scalability-accuracy boundary for approximate knn search in high-dimensional spaces. *PVLDB*, 11(8):906–919, 2018.
- [2] A. Chakrabarti and O. Regev. An optimal randomised cell probe lower bound for approximate nearest neighbour searching. In *FOCS*, pages 473–482, 2004.
- [3] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*, pages 253–262, 2004.
- [4] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*, pages 253–262, 2004.
- [5] J. Gan, J. Feng, Q. Fang, and W. Ng. Locality-sensitive hashing scheme based on dynamic collision counting. In *SIGMOD*, pages 541–552, 2012.
- [6] J. He, S. Kumar, and S.-F. Chang. On the difficulty of nearest neighbor search. In *ICML*, pages 1127–1134, New York, NY, USA, July 2012.
- [7] Q. Huang, J. Feng, Q. Fang, W. Ng, and W. Wang. Query-aware locality-sensitive hashing scheme for lp norm. *VLDB J.*, 26(5):683–708, 2017.
- [8] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *PVLDB*, 9(1):1–12, 2015.
- [9] P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *STOC*, pages 604–613, 1998.
- [10] H. Jégou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011.
- [11] Y. Ke, R. Sukthankar, and L. Huston. An efficient parts-based near-duplicate and sub-image retrieval system. In *ACM Multimedia*, pages 869–876, 2004.
- [12] W. Li, Y. Zhang, Y. Sun, W. Wang, W. Zhang, and X. Lin. Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement. *CoRR abs*, 2016.
- [13] Y. Liu, H. Cheng, and J. Cui. Pqbf: I/o-efficient approximate nearest neighbor search by product quantization. *CIKM*, pages 667–676, 2017.
- [14] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *CoRR*, abs/1603.09320, 2016.
- [15] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *PVLDB*, 8(1):1–12, 2014.
- [16] Y. Tao, K. Yi, C. Sheng, and P. Kalnis. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD*, pages 563–576, 2009.
- [17] J. Wang, W. Liu, S. Kumar, and S. Chang. Learning to hash for indexing big data - A survey. *Proceedings of the IEEE*, 104(1):34–57, 2016.
- [18] J. Wang, J. Zhang, J. Song, N. Sebe, and H. T. Shen. A survey on learning to hash. *IEEE Trans. Pattern Anal. Mach. Intell.*, 40(4):769–790, 2018.